

Capítulo 11

Datasets

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 31/07/2026

Capítulo 11 — Datasets (a virada para texto de verdade)

Objetivo de aprendizagem: montar o **pipeline de dados** completo de um LLM — baixar, limpar, dividir, tokenizar e gravar em formato eficiente. E, com isso, fazer a maior mudança do curso: o modelo deixa de gerar **nomes** e passa a gerar **prosa em português**.

Pré-requisitos: Capítulos 1-10. Em especial o BPE do Capítulo 6, que usamos aqui.

Arquivos: - `prepare_data.py` — o pipeline completo (baixar -> limpar -> dividir -> tokenizar -> gravar) - `dataset.py` — carregamento com `memmap` e formação de batches - `train_text.py` — o Transformer treinado em prosa - `bpe.py` — o tokenizador do Capítulo 6 - `exercicios.md` — exercícios

Aviso sobre a métrica: a loss deste capítulo **não é comparável** com o 1,776 dos capítulos anteriores. Mudou a tarefa (prosa em vez de nomes), mudou a unidade (tokens BPE em vez de caracteres) e mudou o vocabulário (1024 em vez de 27). Prever entre 1024 opções é muito mais difícil que entre 27. O benchmark antigo está **aposentado**; este capítulo estabelece o novo ponto de partida.

1. Por que os dados são o capítulo mais subestimado

Nos capítulos 1 a 10 o dataset era uma lista de nomes que cabia num tensor. Isso permitiu focar na arquitetura — mas escondeu que, num LLM de verdade, **a maior parte do trabalho de engenharia está nos dados**, não no modelo.

O modelo do GPT-3 cabe em algumas centenas de linhas. O corpus dele tem ~570 GB filtrados a partir de muito mais. A proporção de esforço é essa mesma.

Este capítulo monta o pipeline inteiro, e cada etapa tem uma armadilha:

```
baixar -> limpar -> DIVIDIR -> tokenizar -> gravar em binário
                ^  ^  ^  ^
                |  |  |  |
                |  |  |  |
                lixo que vazamento vazamento memória
                o modelo por tema de vocabulário
                aprenderia
```

2. O corpus: Machado de Assis

Escolhemos as obras de **Machado de Assis** (1839-1908), obtidas do **Project Gutenberg**. Três razões:

1. **É português literário de verdade** — prosa complexa, vocabulário rico, pontuação variada. Bem mais difícil (e mais interessante) que nomes.
2. **A licença é limpa.** Machado morreu em 1908; a obra está em domínio público. Num curso que ensina a treinar modelos, ignorar a procedência dos dados seria um mau exemplo — a origem e a

licença do corpus são parte do trabalho.

3. **É cultural e linguisticamente nosso**, o que combina com um curso em português.

```
Dom Casmurro 410037 -> 378215 chars ( 7.8% removido)
Memorias Postumas de Bras Cubas 396533 -> 363830 chars ( 8.2% removido)
Quincas Borba 484321 -> 453033 chars ( 6.5% removido)
Esau e Jaco 444833 -> 414801 chars ( 6.8% removido)
Memorial de Aires 316580 -> 284854 chars (10.0% removido)
```

Limpeza: o que aqueles 7-10% eram

Os arquivos do Gutenberg vêm com cabeçalho de licença, créditos e metadados **em inglês**. Deixar isso no corpus ensinaria o modelo a escrever termos de licença americanos no meio de um romance brasileiro — lixo competindo com o texto que interessa. O `prepare_data.py` corta pelos marcadores:

```
inicio = re.search(r"\*\*\s*START OF TH[EIS] PROJECT GUTENBERG EBOOK.*?\*\*\s*",
texto)
fim = re.search(r"\*\*\s*END OF TH[EIS] PROJECT GUTENBERG EBOOK.*?\*\*\s*", texto)
```

Deduplicação

Corpus real tem repetição: sumários, cabeçalhos, trechos citados. Texto duplicado faz o modelo **memorizar** em vez de generalizar, e contamina a avaliação se a mesma passagem cair nos dois lados da divisão. Removemos parágrafos idênticos (com mais de 40 caracteres, para não apagar repetições legítimas como "— Sim.").

3. A divisão: onde quase todo mundo erra

Duas decisões que parecem detalhes e não são.

Divida por documento, não por linha sorteada

O jeito errado, e muito comum: juntar tudo, embaralhar os parágrafos, separar 15% para validação. Parece razoável e é **vazamento**.

Se você embaralhar os parágrafos de *Dom Casmurro*, o modelo treina no capítulo inteiro e é avaliado em frases **do mesmo capítulo** — mesmos personagens, mesmo assunto, mesmo estilo, às vezes as mesmas construções. A loss de validação fica boa e **não significa nada**.

O jeito certo é separar por **obra inteira**:

```
treino: 4 obras, 1609699 chars (Dom Casmurro, Brás Cubas, Quincas Borba, Esaú e Jacó)
val : 'Memorial de Aires', 284854 chars
proporcao: 15.0% para validacao
```

Memorial de Aires é um livro que o modelo **nunca vê**. Avaliar nele mede o que a gente quer medir: o modelo aprendeu **português e o estilo de Machado**, ou decorou frases?

Quanto isso muda na prática — medido

A solução do exercício E2 quantifica o vazamento sem precisar treinar nada: conta quantas sequências de N tokens do conjunto de validação **já aparecem literalmente** no treino.

n-grama	Divisão correta	Divisão errada	Razão
3 tokens	26,74%	31,10%	1,2x
5 tokens	2,82%	4,86%	1,7x
8 tokens	0,24%	0,59%	2,4x
12 tokens	0,13%	0,16%	1,3x

Os n-gramas **curtos** aparecem muito nos dois casos — isso é só a língua portuguesa ("de que a", "não se"), não vazamento. Os **longos** é que denunciam: uma sequência específica de 8 tokens aparecer nos dois lados não acontece por acaso.

O efeito é consistente (a divisão errada perde em todos os tamanhos), mas seja honesto sobre a **magnitude**: aqui ele é **moderado**, não dramático. Duas razões, e ambas ensinam algo:

1. Nosso embaralhamento é de blocos de 512 tokens. Trechos diferentes do mesmo livro ainda são texto diferente. Com embaralhamento por **frase**, ou com documentos duplicados, o vazamento seria bem maior.
2. Este corpus é de **um autor só**. Machado escreve parecido em todos os livros, então até a divisão honesta tem sobreposição real de estilo. Isso é informação legítima.

Em corpus da web — onde o mesmo artigo aparece copiado em dezenas de sites — o efeito é muito maior. É por isso que **deduplicação** é passo obrigatório nos pipelines de verdade, e por que laboratórios publicam quanto do benchmark vazou para o treino.

Divida ANTES de tokenizar

Sutil e importante: o tokenizador é **treinado nos dados**. Se ele vir o texto de validação, ele aprende o vocabulário de lá — nomes de personagens, expressões específicas — e a avaliação fica otimista. É vazamento por outro caminho.

```
tok.train(texto_treino[:BYTES_TREINO_TOKENIZADOR], VOCAB_SIZE)
# ^^^^^^^^^^^^^ só o treino, nunca o de validação
```

4. Tokenização: o BPE aprende o domínio

Usamos o BPE do Capítulo 6, com vocabulário de 1024. Veja o que ele aprendeu deste corpus:

```
tokens mais longos: 'José Dias ', 'ima Justin', 'que não ', 'minha mã',  
'Capitú, ', 'os olhos ', 'da minha ', 'Capitú '
```

Ele descobriu sozinho os **personagens de Dom Casmurro** — José Dias, Capitú, Prima Justina — e os transformou em tokens únicos. Ninguém disse a ele que são nomes; ele apenas notou que essas sequências de bytes se repetem muito.

Isso é ótimo para este corpus e revelador sobre a natureza dos tokenizadores:

Um tokenizador é um modelo estatístico do corpus em que foi treinado. Ele comprime muito bem o que conhece, e mal o que não conhece. Um tokenizador treinado em Machado gastaria muitos tokens num artigo de medicina — e vice-versa. Foi a mesma lição do Capítulo 6, agora com consequência prática.

Medindo a compressão no corpus inteiro:

Split	Caracteres	Tokens	Compressão
treino (4 obras)	1.609.699	621.134	2,64x
validação (1 obra)	284.854	108.628	2,68x

A compressão se sustenta no texto de validação (2,68x contra 2,64x) — o tokenizador aprendeu o **padrão** do português literário, não decorou as frases do treino. Compare com o Capítulo 6, onde a compressão caiu de 2,24x para 1,20x ao sair do domínio: aqui não cai, porque validação e treino são o mesmo domínio (prosa do mesmo autor), só que textos diferentes.

Detalhe de implementação: o BPE em Python puro é lento, então o treinamos num subconjunto de 300 mil caracteres (34 segundos) e o **aplicamos** ao corpus inteiro. E a tokenização é feita parágrafo a parágrafo, não no texto todo de uma vez — cada chamada ao **encode** percorre a sequência repetidamente, então textos curtos são muito mais rápidos. Foram 67s para o treino e 11s para a validação.

5. Gravar em binário: **uint16** e **memmap**

Duas decisões de formato que parecem prematuras e não são.

Por que **uint16?** O vocabulário tem 1024 tokens, então cada token cabe em 16 bits. Guardar em **int64** (o padrão do PyTorch) gastaria **4x mais espaço** para representar exatamente a mesma informação.

Por que binário puro, sem cabeçalho? Porque permite **np.memmap**:

```
dados = np.memmap("treino.bin", dtype=np.uint16, mode="r")
```

O **memmap** não lê o arquivo — ele mapeia o arquivo no espaço de endereçamento, e o sistema operacional traz do disco apenas as páginas efetivamente tocadas. Medindo:

```
np.fromfile (carrega tudo): 0.5 ms, 1.2 MB na RAM
np.memmap (so' mapeia) : 0.4 ms, ~0 MB na RAM
```

E a formação de batches é rápida o bastante para nunca ser o gargalo:

batch × block	Tempo	Vazão
32 × 64	0,89 ms	2,29 M tokens/s
64 × 128	1,65 ms	4,96 M tokens/s
128 × 256	3,36 ms	9,76 M tokens/s

Se o carregamento de dados for mais lento que o passo de treino, a GPU fica **esperando** — e você paga por uma placa ociosa. É por isso que frameworks usam **DataLoader** com workers paralelos e *prefetch*. Aqui, como cada batch é só a leitura de fatias contíguas, o custo é desprezível perto do forward/backward.

Com um corpus deste tamanho a diferença é irrelevante. A questão é o que acontece com 100 GB: o **fromfile** simplesmente não roda, e o **memmap** continua funcionando igual. Escrever assim desde o início custa nada e evita reescrever o pipeline depois.

6. A mudança no formato do treino

Aqui há uma diferença importante em relação aos capítulos 3-7.

Antes: cada exemplo era uma janela de contexto com preenchimento, e havia **um** alvo (o próximo caractere). Um batch de 64 exemplos gerava 64 previsões.

Agora: o texto é um **fluxo contínuo**, e cada posição tem um alvo. O truque é elegante:

```
x = tokens[i : i + block_size] # entrada
y = tokens[i + 1 : i + block_size + 1] # alvo = a entrada DESLOCADA DE UM
```

O alvo é o próprio texto deslocado de um token. E o modelo prevê em **todas** as posições:

```
logits = self.lm(self.ln(x)) # (B, T, vocab)
loss = F.cross_entropy(logits.view(-1, vocab_size), alvos.view(-1))
```

Com **batch_size=32** e **block_size=128**, isso dá **4.096 previsões por batch** em vez de 32 — o sinal de treino fica 128x maior **pelo mesmo custo de forward**. É assim que LLMs são treinados de verdade.

E note o que isso significa: não existe rótulo humano nenhum aqui. O texto já é a resposta. É por isso que se chama aprendizado **auto-supervisionado**, e é por isso que dá para treinar com a internet inteira sem ninguém anotar nada.

7. Resultados: o modelo escreve prosa

O `train_text.py` treina um Transformer de **2.196.352 parâmetros** (4 blocos, 6 cabeças, `n_embd=192`, contexto de **128 tokens**) por 3.000 passos — cerca de 18 minutos na CPU.

```
passo 0 | treino 7.1026 | val 7.1043 | 3s
passo 500 | treino 4.3011 | val 4.5850 | 193s
passo 1000 | treino 3.5819 | val 4.0891 | 374s
passo 1500 | treino 3.2023 | val 3.9380 | 552s
passo 2000 | treino 2.9862 | val 3.9176 | 730s
passo 2500 | treino 2.8253 | val 3.8969 | 911s

FINAL: treino 2.7569 | validacao 3.9450
perplexidade de validacao: 51.7
```

A **perplexidade** é `exp(loss)`, e tem leitura intuitiva: é o número médio de opções entre as quais o modelo está "em dúvida" a cada token. Um modelo que chutasse uniformemente entre 1024 tokens teria perplexidade 1024. O nosso está em **51,7** — ele reduziu o espaço de dúvida em 20 vezes.

O overfitting é real, e a conta explica

Repare no que aconteceu a partir do passo 1500: a loss de **treino** continua caindo (3,20 -> 2,76) enquanto a de **validação** praticamente estaciona (3,94 -> 3,95). Isso é *overfitting*, o mesmo fenômeno do Capítulo 3 — e desta vez a causa é fácil de calcular:

Grandeza	Valor
Parâmetros do modelo	2.196.352
Tokens de treino	621.134
Tokens por parâmetro	0,28

Temos **mais parâmetros que tokens de treino**. A regra prática mais conhecida (*Chinchilla*, DeepMind 2022) sugere cerca de **20 tokens por parâmetro** para um treino equilibrado — estamos ~70x abaixo disso. Com essa proporção, memorizar é mais fácil que generalizar, e o modelo faz o que é mais fácil.

Duas saídas, e é útil saber qual escolher: menos parâmetros (modelo menor) ou mais dados. O exercício E7 explora a segunda — as obras completas de Machado no Gutenberg são 12, não 5.

O que ele escreve

Este é o ponto do capítulo. Com temperatura 0,8:

de cós. Leito de Baptista tinha
de fazer um grande e outro lado da beleza da devida; era a nossa gente;
a dissesse a figura de seis velhos, eu represso para os dias de
roupa da liberdade de Fulher e adiante nas caras, mas os homens, a primeira
mão; a filha não é só um collegio, o imperiogro, nem as alegres
de uma boca philosophias na vespera á mãe que a filha não aliás
instituição. Um collo jura

Não é Machado, e boa parte não faz sentido semântico — esperado para 2,2 milhões de parâmetros treinados em 1,6 MB de texto. Mas olhe **o que já está lá**:

- **é português**, com concordância local plausível ("a filha não é só um colégio")
- **pontuação correta**: vírgulas, ponto e vírgula, pontos finais nos lugares certos
- **estrutura de diálogo**, com o travessão que Machado usa (--Sim, ...)
- **quebras de parágrafo** e ritmo de frase
- **a ortografia de época** do corpus (beleza, collegio, philosophias) — ele aprendeu a escrever como se escrevia em 1880, porque é isso que os dados ensinam

Compare com o que o mesmo curso gerava antes:

Capítulo	Saída
01 (bigrama)	cexzma, zktahwelo
05 (Transformer)	jandir, valdinia
11 (prosa)	frases em português, com pontuação e diálogo

O modelo não ficou "mais inteligente" — a arquitetura é a mesma do Capítulo 5, só um pouco maior. **Os dados é que mudaram.** É a demonstração mais direta possível da tese deste capítulo.

8. Resumo do capítulo

- **Dados são a maior parte do trabalho** de um LLM, e o capítulo mais subestimado.
- **Procedência importa**: usamos domínio público (Machado de Assis, Gutenberg) e declaramos a fonte. Num curso sobre treinar modelos, isso é parte do ofício.
- **Limpeza**: cabeçalhos de licença em inglês seriam aprendidos como texto — 7-10% do arquivo era isso.
- **Deduplicação** evita memorização e contaminação da avaliação.
- **Divida por documento, não por linha sorteada** — senão o modelo é avaliado em texto quase idêntico ao que treinou.
- **Divida antes de tokenizar** — o tokenizador aprende com os dados, e vazaria também.
- **O BPE aprende o domínio**: descobriu 'José Dias ' e 'Capitú, ' como tokens únicos.

- `uint16 + memmap`: 4x menos espaço e leitura sem carregar na RAM — requisito, não otimização, quando o corpus cresce.
- **Prever em todas as posições** multiplica o sinal de treino por `block_size` de graça.

O que vem no Capítulo 12

Temos um modelo que escreve prosa — e que é **lento** para gerar, porque recalcula tudo a cada token novo. No **Capítulo 12 — Inference I: KV-cache** vamos guardar as chaves e valores já calculados e acelerar a geração em várias vezes, sem mudar uma vírgula do que o modelo produz.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 11 (Datasets)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Rode `python prepare_data.py` uma vez antes dos exercícios (ele baixa e prepara o corpus; os livros ficam em cache, então rodar de novo é rápido).

E1 — Por que essa ordem? (aquecimento)

Sem rodar: 1. Por que o tokenizador é treinado **só** no texto de treino? O que exatamente vazaria se ele visse o texto de validação? 2. Por que dividimos **por obra** e não sorteando parágrafos? Dê um exemplo concreto do que o modelo "veria de graça" na divisão errada. 3. Por que gravamos os tokens como `uint16` e não `int64`? Quanto de espaço isso poupa?

E2 — Faça a divisão errada de propósito (importante)

Modifique o `prepare_data.py` para juntar **todas as 5 obras**, embaralhar os parágrafos e separar 15% aleatoriamente para validação. 1. Treine o modelo com essa divisão. A loss de validação fica **melhor** ou pior? 2. Ela ficou melhor? Isso significa que o modelo é melhor? Explique o que aconteceu. 3. Esse é um dos erros mais comuns em ML aplicado — e ele faz o modelo parecer bom no relatório e falhar na prática. Como você detectaria isso numa revisão de código?

Solução de referência: [solucoes/e2_vazamento.py](#).

E3 — O tokenizador é do domínio

O BPE treinado neste corpus aprendeu tokens como `'José Dias '`, `'Capitú, '` e `'minha mãe'`. 1. Liste os 20 tokens mais longos aprendidos. Quantos são nomes de personagens? 2. Isso é bom ou ruim? (Pense: e se você usasse este tokenizador para tokenizar um texto de medicina?) 3. Treine um BPE no corpus de **nomes** do Capítulo 6 e tokenize um trecho de Machado com ele. Quantos tokens a mais são necessários?

E4 — Tamanho do contexto (o mais instrutivo do capítulo)

No `train_text.py`, varie o `block_size`: 32, 128 e 256.

1. **Preveja primeiro.** Prosa tem dependências longas (concordância, referência a personagens) que nomes de 7 letras não têm. Qual `block_size` deve vencer? Agora meça, com **poucos passos** (400 serve) — bateu com a previsão?
2. **Meça de novo com o orçamento cheio** (3.000 passos). O ranking é o mesmo? Olhe também a diferença entre a loss de treino e a de validação em cada configuração.
3. O custo da atenção cresce com T^2 . Dobrar o contexto quadruplica o tempo por passo?

4. Com o número de **passos** fixo, quantas vezes cada configuração passa pelo corpus inteiro? (Conta: $\text{batch} \times \text{block} \times \text{passos} \div \text{tokens do corpus}$.) O que isso explica sobre o que você viu no item 2?

E5 — Prever em todas as posições

Este capítulo mudou o modelo para prever em **todas** as posições, não só na última. 1. Quantas previsões por batch o modelo faz agora, contra antes? (Use `batch_size` e `block_size`.) 2. Modifique o `forward` para usar só a última posição (como nos capítulos 5-7) e treine com o mesmo orçamento de passos. A loss final é pior? Muito? 3. O custo do forward mudou? Se não mudou, o que isso diz sobre "aproveitar o cálculo"?

E6 — Dados sintéticos (desafio)

O syllabus deste capítulo inclui **geração de dados sintéticos**. Use o modelo treinado para gerar 200 KB de texto, e depois treine um **segundo** modelo só com esse texto gerado. 1. O segundo modelo chega perto do primeiro? Ou fica pior? 2. Esse fenômeno tem nome na literatura (*model collapse*). Explique com os seus números por que treinar em dados gerados pelo próprio modelo degrada a qualidade. 3. Em que situações dados sintéticos **ajudam** de verdade? (Dica: pense em tarefas onde é possível **verificar** a resposta, como matemática ou código.)

E7 — Escalando o corpus (desafio)

O `prepare_data.py` usa 5 obras. A lista completa de Machado no Project Gutenberg tem 12. 1. Acrescente mais obras e refaça o preparo. Como muda a loss de validação? 2. Mantenha o modelo do mesmo tamanho. A partir de quanto texto o ganho satura? 3. Relacione com o Capítulo 3: lá, aumentar os dados de 155 para 64 mil nomes resolveu o *overfitting*. A mesma lógica vale aqui? Compare as losses de treino e validação.